

Week 3 Module – Programming Practice (3rd Year)

Date: 29/06/2026

Language Used: JAVA

Pdf follows: handwritten code with logic and followed by the respective outputs.

Submission details:---

Name : Sakshi das

Roll number : 2404921520194

Task completed on : 05/07/2026 – Sunday

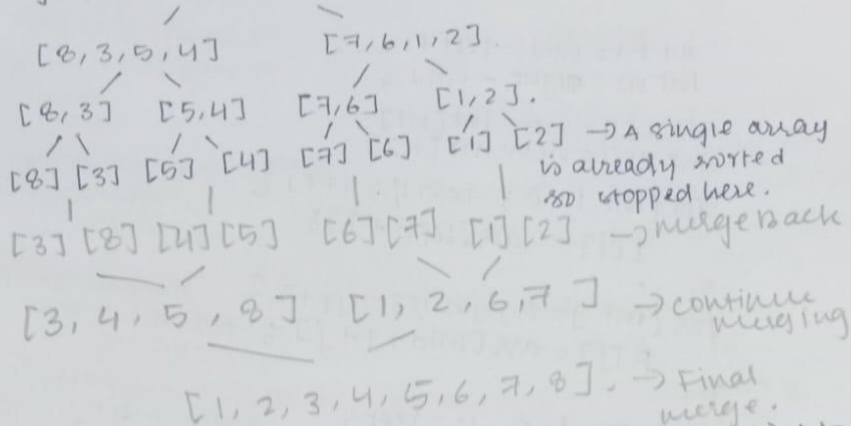
Week 8

Date: 29/06/2026.

Merge sort concept :-

Divide $\rightarrow$ sort halves $\rightarrow$ merge sorted halves.

eg: [8, 3, 5, 4, 7, 6, 1, 2].



So, basically sorted left halves $\rightarrow$ sorted right halves $\rightarrow$ merge them.

Concept of Recursion helps here.

Q17 // merge sort.

eg :- i/p : 6

450 1200 800 650 300 1500

o/p : 300 450 650 800 1200 1500.

Java code :-

```
import java.util.*;
public class main {
    static void merge (int arr[], int left, int mid, int right) {
        //Function to merge two sorted sub-arrays
        int n1 = mid - left + 1; // size of left & right subarrays.
        int n2 = right - mid;
        int L[] = new int[n1];
        int R[] = new int[n2];
        for (int i = 0; i < n1; i++) { // copied data to left temporary array "L"
            L[i] = arr[left + i];
        }
        for (int j = 0; j < n2; j++) { // copied data to right temporary array "R"
            R[j] = arr[mid + 1 + j];
        }
        int i = 0, j = 0, k = left; // k will originally point original array.
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                arr[k] = L[i];
                i++;
            } else {
                arr[k] = R[j];
                j++;
            }
            k++;
        }
        while (i < n1) {
            arr[k++] = L[i++];
        }
    }
}
```

← from here we ~~copy~~ compare elements and place smaller one in original array.

```
while (j < n2) {
```

```
    arr[k++] = R[j++];
```

```
} // close of while loop
```

```
} // close of merge function.
```

```
static void mergesort (int arr[], int left, int right) {
```

* When subarray has one element → stop

```
    if (left < right) {
```

```
        int mid = (left + right) / 2; // mid
```

```
        mergesort (arr, left, mid); // sort left
```

```
        mergesort (arr, mid + 1, right); // sort right
```

```
        merge (arr, left, mid, right); // merge both
```

```
    } // close of if condition.
```

```
} // close of Recursive merge sort function.
```

```
public static void main (String[] args) {
```

```
    Scanner sc = new Scanner (System.in);
```

```
    int n = sc.nextInt(); // take i/p size
```

```
    int arr[] = new int[n]; // create array
```

```
    for (int i = 0; i < n; i++) { // take i/p elements
```

```
        arr[i] = sc.nextInt();
```

```
    }
```

```
    mergesort (arr, 0, n - 1); // sort using mergesort
```

```
    for (int num : arr) {
```

```
        System.out.print (num + " ");
```

```
    } // close of for loop
```

```
} // close of main function.
```

```
} // close of class.
```

Key point :- `for (int num : arr)` means →

for every element in the array arr, store in variable num & execute the body.

```
for (int i = 0; i < arr.length; i++) {
```

```
    int num = arr[i];
```

```
    System.out.print (num + " ");
```

```
} // in simple terms.
```

Output of type-1 Q-1.

```
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA> cd "c:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalT
raining\week3\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
6
450
1200
800
650
300
1500
300 450 650 800 1200 1500
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3>
```

Ln 94, Col 2 Spaces: 4 UTF-8 CRLF { } Java Go Live Prettier

Q2) eg:- 12 15 18 22 25 30 40.

Q2) eg:- i/p:- 7

12 25 18 30 15 22 40.

o/p:- 12 15 18 22 25 30 40

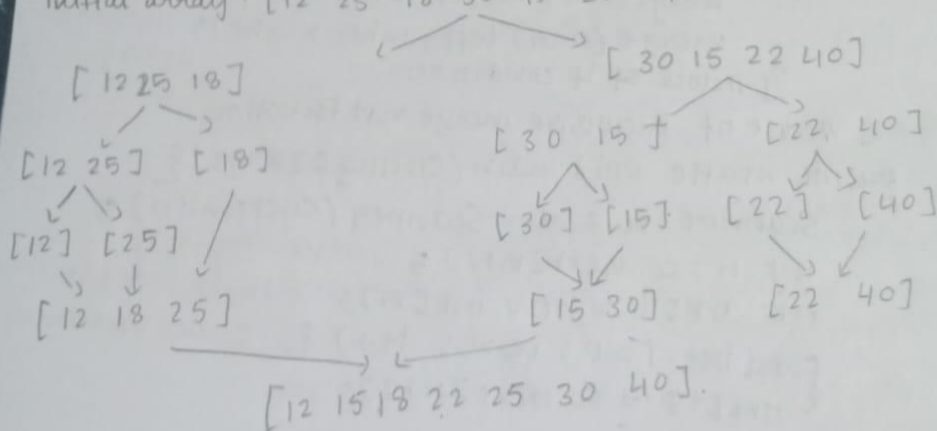
median = 20

orders above median = 3

difference = 28.

dry run for eg:-

initial array [12 25 18 30 15 22 40]



Median :- $n=7$ (odd)

1) median = $\text{arr}[N/2]$

= $\text{arr}[3] = 22$.

2) count of orders > median (22).

2) elements greater than 22 → 25, 30, 40.

⇒ count = 3

max = 40

min = 12

} difference ⇒ $40 - 12 = 28$

- (1) Read N
- (2) Apply merge sort to sort the array in *Ascending order*.
- (3) Display sorted array.
- (4) Finding median:-
 - $N(\text{odd}) \Rightarrow \text{arr}[N/2]$
 - $N(\text{even}) \Rightarrow \text{arr}[N/2 - 1]$ and $\text{arr}[N/2]$.
- (5) count the no. greater than median.
- (6) diff. b/w max & min elements.
- (7) Display the results.

Java code :-

```
import java.util.*;
public class main {
    // Merge two sorted subarrays.
    static void merge(int arr[], int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;
        int L[] = new int[n1];
        int R[] = new int[n2];
        // copy data.
        for (int i = 0; i < n1; i++) {
            L[i] = arr[left + i];
        }
        for (int j = 0; j < n2; j++) {
            R[j] = arr[mid + 1 + j];
        }
        int i = 0;
        int j = 0;
        int k = left;
```

// merge the temporary arrays.

while (i < n1 && j < n2) {

if (L[i] <= R[j]) {

arr[k] = L[i];

i++;

}

else {

arr[k] = R[j];

j++;

}

k++;

}

while (j < n2) {

arr[k] = R[j];

j++;

k++;

}

// close of merge function.

static void mergesort(int arr[], int left, int right) {

if (left < right) {

int mid = (left + right) / 2;

mergesort(arr, left, mid);

mergesort(arr, mid + 1, right);

mergesort(arr, left, mid, right);

}

// close of mergesort function.

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

int n = sc.nextInt();

int arr[] = new int[n];

for (int i = 0; i < n; i++) {

arr[i] = sc.nextInt();

}


```

mergesort(arr, 0, n-1);
for(int num : arr) {
    system.out.print(num + " ");
}
system.out.println();

//find median
int median;
if (n%2 == 1) {
    median = arr[n/2];
}
else {
    median = (arr[n/2 - 1] + arr[n/2]) / 2;
}
system.out.println("median" + median);

int count = 0;
for(int num : arr) {
    if (num > median) {
        count++;
    }
}
system.out.println("Orders above median:"
                    + count);

int difference = arr[n-1] - arr[0];
system.out.println("Difference : " + difference);
} //close of main function
} //close of class.

```

```
PS C:\Users\saksh\OneDrive\Desktop\coding> cd "c:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3\" ; if ($?) { javac next.java } ; if ($?) { java next }
7
12
25
18
30
15
22
40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3>
```

Output of type-1 Q-2.

Topic 2: Quick Sort.

Concept:-

It divides array recursively.

It follows divide and conquer strategy.

eg: 5 3 8 1 2 7 6.

5 3 1 8 1 2 7 6 ...

Steps:-

i) Choose a pivot.

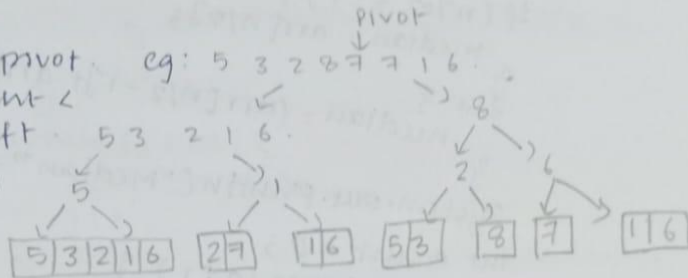
eg: 5 3 2 8 7 7 1 6.

ii) All element <

pivot → go left.

and > pivot,

go right.



iii) Recurse on Left & Right sub-arrays.

Q1) eg:- i/p → 6
120 45 78 200 60 95

o/p → 45 60 78 95 120 200.

code:-

```
import java.util.*;
public class main3 {
    static int partition (int arr[], int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}
```

```
int temp = arr[i+1];  
arr[i+1] = arr[high];  
arr[high] = temp;
```

```
return i+1;
```

3. close of partition function.

```
static void quicksort (int arr[], int low,  
int high) {
```

```
if (low < high) {
```

```
int pi = partition (arr, low, high);
```

```
quicksort (arr, low, pi-1);
```

```
quicksort (arr, pi+1, high);
```

```
}
```

3. close of quicksort function.

```
public static void main (String [] args) {
```

```
Scanner sc = new Scanner (System.in);
```

```
int n = sc.nextInt();
```

```
int arr[] = new int [n];
```

```
for (int i=0; i<n; i++) {
```

```
arr[i] = sc.nextInt();
```

```
}
```

```
quicksort (arr, 0, n-1);
```

```
for (int num : arr) {
```

```
System.out.print (num + " ");
```

```
}
```

3. close of main function.

3. close of class.



The image shows a screenshot of a Visual Studio Code (VS Code) terminal window. The terminal is running a Java program. The command prompt shows the user is in the directory `C:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3`. The command executed is `cd "c:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3" ; if ($?) { javac Main3.java } ; if ($?) { java Main3 }`. The output of the program is displayed on the next line: `6`, `120`, `45`, `78`, `200`, `60`, `95`, and `45 60 78 95 120 200`. The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. On the right side of the terminal window, there is a sidebar with icons for powershell, powershell, Code, and Code week3.

```
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA> cd "c:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3" ; if ($?) { javac Main3.java } ; if ($?) { java Main3 }
6
120
45
78
200
60
95
45 60 78 95 120 200
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3>
```

Output of type-2 Q-1.

Q2) eg). i/p = 8.

500 1200 700 2000 900 1500 3000 2500

o/p =
3000 2500 2000 1500 1200 900 700 500

Top 5 = 3000 2500 2000 1500 1200

Avg Top 5 = 2040

values $\geq$ Above overall Avg = 3.

500 1200 700 2000 900 1500 3000 2500

↳ Descending order : 3000 2500 2000 1500 1200
900 700 500.

↳ Top 5 :
3000
2500
2000
1500
1200
→ Average of Top 5
$$= \frac{3000 + 2500 + 2000 + 1500 + 1200}{5}$$
$$= 2040.$$

⇒ overall average

⇒
$$\frac{3000 + 2500 + 2000 + 1500 + 1200 + 900 + 700 + 500}{8}$$

⇒
$$\frac{12300}{8} = 1537.5$$

values above
avg →

3000
2500
2000

count = 3

code :-

```
import java.util.*;  
public class main4 {  
    static int partition (long arr[], int low, int high) {  
        int long pivot = arr[high];  
        int i = low - 1;  
        for (int j = low; j < high; j++) {  
            if (arr[j] >= pivot) {  
                i++;  
                long temp = arr[i];  
                arr[i] = arr[j];  
                arr[j] = temp;  
            }  
        }  
        long temp = arr[i+1];  
        arr[i+1] = arr[high];  
        arr[high] = temp;  
        return i+1;  
    }  
    static void quicksort (long arr[], int low, int high) {  
        if (low < high) {  
            int pi = partition (arr, low, high);  
            quicksort (arr, low, pi-1);  
            quicksort (arr, pi+1, high);  
        }  
    }  
    public static void main (String[] args) {  
        Scanner sc = new Scanner (System.in);  
        int n = sc.nextInt();  
        long arr[] = new long [n];  
        long totalSum = 0;
```

```

For (int i=0; i<n; i++) {
    arr[i] = sc.nextLong();
    totalSum += arr[i];
}
//descending
//order
//sorting.
amichSort(arr, 0, n-1);
for (long num : arr) {
    System.out.print(num + " ");
}
System.out.println();

```

//displaying
Top 5.

```

System.out.print("Top 5: ");
long top5Sum = 0;
for (int i=0; i<5; i++) {
    System.out.print(arr[i] + " ");
    top5Sum += arr[i];
}

```

//avg of Top 5

```

double top5Average = top5Sum / 5.0;
if (top5Average == (long)top5Average) {
    System.out.println("Average of  
Top 5: " + (long)top5Average);
}
else {

```

```

    System.out.printf("Average of Top 5:  
%0.2f \n", top5Average);
}

```

//overall
Ans. double overallAverage = totalSum/(double) n;

- int count = 0;

```
for (long value : arr) {  
    if (value > overallAverage) {  
        count++;  
    }  
}
```

System.out.println("values above
overall average : " + count);

}

3.

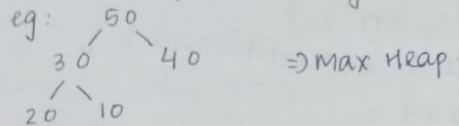
*used long because the
constraints show up to 10^9 → and summing many
values
↓
this will make int to
overflow.

```
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3> cd "c:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3\" ; if ($?) { javac Main4.java } ; if ($?) { java Main4 }
8
500
1200
700
2000
900
1500
3000
2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3>
```

Output of type-2 Q-2.

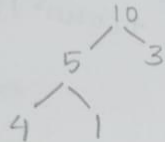
Type 3: Heap Sort.

Heap is a complete Binary tree.

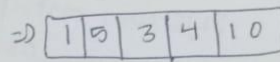


eg:- Array:

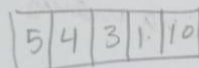
4	10	3	5	1
---	----	---	---	---



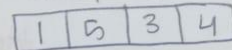
⇒ move the largest element to the end



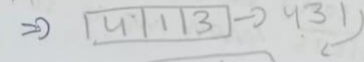
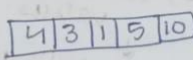
↓
Heap again



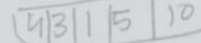
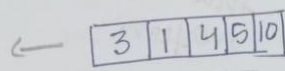
⇒ 5 4 3 1



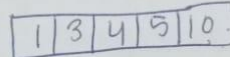
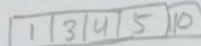
↓
swap ⇒



3 1



↘ 1 3 ⇒



Q11.
eg:- 1) i/p :- 6 850 1200 950 600 1500 1000

o/p ⇒ 600 850 950 1000 1200 1500.

Code:-

```
import java.util.*;
public class main {
    static void heaped (int arr[], int n, int i) {
        int largest = i;
        int left = 2*i + 1;
        int right = 2*i + 2;
```

```

    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
    }
    heaped(arr, n, largest);
}

```

```

3. static void heapsort(int arr[]) {
    int n = arr.length;
    for (int i = n/2 - 1; i >= 0; i--) {
        heaped(arr, n, i);
    }
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        heaped(arr, i, 0);
    }
}

```

```

3. public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
}

```

```

for(int i=0; i<n; i++) {
    arr[i] = sc.nextInt();
}
heap
heapsort(arr);
for (int num: arr) {
    System.out.print(num + " ");
}
}
}

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA> cd "c:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalT
raining\week3\" ; if ($?) { javac Main5.java } ; if ($?) { java Main5 }
6
850
1200
950
600
1500
1000
600 850 950 1000 1200 1500
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3>

```

Output of type-3 Q-1.

Q2) eg:- i/p $\Rightarrow$ 8

12 7 15 9 20 5 10 8.

O/p $\Rightarrow$ 5 7 8 9 10 12 15 20.

Fastest: 5

Slowest: 20

Average: 10.75

Cases faster than Average: 5

Percentage: 62.50%.

code :-

```
import java.util.*;
public class main6 {
    static void heaped (int arr[], int n, int i) {
        int largest = i;
        int left = 2*i + 1;
        int right = 2*i + 2;
        if (left < n && arr[left] > arr[largest]) {
            largest = left;
        }
        if (right < n && arr[right] > arr[largest]) {
            largest = right;
        }
        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;
            heaped (arr, n, largest);
        }
    }

    static void heapsort (int arr[]) {
        int n = arr.length;
        for (int i = n/2 - 1; i >= 0; i--) {
            heaped (arr, n, i);
        }
        for (int i = n-1; i > 0; i--) {
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;
            heaped (arr, i, 0);
        }
    }
}
```



```

public static void main (String[] args) {
    Scanner sc = new Scanner (System.in);
    int n = sc.nextInt();
    int arr[] = new int [n];
    int sum = 0;
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
        sum += arr[i];
    }
    heapSort (arr);
    System.out.println ("Sorted Response
                        Times: ");
    for (int num : arr) {
        System.out.print (num + " ");
        System.out.println ();
    }
    System.out.println ("Fastest: " + arr[0]);
    System.out.println ("Slowest: " +
                        arr[n-1]);
    double avg = (double) sum / n;
    System.out.printf ("Average: %.2f\n",
                        avg);
    int count = 0;
    for (int num : arr) {
        if (num < avg) {
            count++;
        }
    }
}

```

3.
 *printf
 instead
 and

system.out.println("cases faster than

Average : " + count) ;

double percentage = (count * 100.0) / n ;

system.out.printf("Percentage : %0.2f%%\n",

percentage) ;

3

3.

*printf used to match to question.
instead of 10.750000 $\Rightarrow$ 10.75
and instead of 62.5% $\Rightarrow$ 62.50%.

```
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3> cd "c:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3\" ; if ($?) { javac Main6.java } ; if ($?) { java Main6 }
8
12
7
15
9
20
5
10
8
Sorted Response Times:
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
PS C:\Users\saksh\OneDrive\Desktop\coding\JAVA\TechnicalTraining\week3>
```

Output of type-3 Q-2.